

BRAC's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 7

Roll no: 62 PRN No: 12411517 Name of Student: Akshada Lohakare

Subject Name : Deep Learning

Problem Statement : Implement transfer learning using pre-trained AlexNet, VGG16, ResNet50, and EfficientNetB0 models for image classification, and compare their performance

Algorithm :

1. Load the Dataset: Load the Butterfly dataset and select five butterfly classes for classification.
2. Preprocess the Images: Resize all images to 224×224 pixels, convert them into tensors, and normalize them using ImageNet mean and standard deviation values.
3. Split the Dataset: Divide the dataset into 70% training data and 30% testing data using stratified splitting.
4. Load Pretrained Models: Load AlexNet, VGG16, ResNet50, and EfficientNetB0 with pretrained ImageNet weights.
5. Freeze Pretrained Layers: Freeze the existing convolutional layers so that the learned ImageNet features are not modified during training.
6. Modify Classifier: Replace the original ImageNet classification layer of each model with a new classification layer having five output classes.
7. Train the Models: Train the new classification layers using the butterfly training images with the Adam optimizer and cross entropy loss.
8. Evaluate the Models: Use the testing dataset to generate predictions and calculate the classification accuracy of each pretrained model.
9. Compare Performance: Store the test accuracy of all four models and compare their performance using a bar graph.

10. Analyze Results: Compare the obtained accuracies to understand the performance of different pretrained CNN architectures for butterfly image classification.

Code :

```
# Import required libraries

import os
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

from PIL import Image
from sklearn.model_selection import train_test_split

from torchvision import models, transforms
from torch.utils.data import Dataset, DataLoader

# Check whether GPU is available

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Device:", device)

# Load the butterfly dataset

df = pd.read_csv("Training_set.csv")

print("\nDataset Preview")
print(df.head())

print("\nDataset Columns")
print(df.columns)

# Select five butterfly classes
```

```
classes = df["label"].value_counts().head(5).index

df = df[df["label"].isin(classes)]

# Select a maximum of one hundred images from each class

df = df.groupby("label").head(100)

print("\nImages in each class")
print(df["label"].value_counts())

print("\nTotal Images:", len(df))

# Create numerical labels for the classes

classes = sorted(df["label"].unique())

class_to_idx = {
    name: i for i, name in enumerate(classes)
}

print("\nClasses")
print(classes)

# Split the dataset into training and testing data

train_df, test_df = train_test_split(
    df,
    test_size=0.30,
    stratify=df["label"],
    random_state=42
)

print("\nTraining Images:", len(train_df))
print("Testing Images:", len(test_df))

# Define image preprocessing

transform = transforms.Compose([

    transforms.Resize((224, 224)),
```

```

        transforms.ToTensor(),

        transforms.Normalize(
            [0.485, 0.456, 0.406],
            [0.229, 0.224, 0.225]
        )
    ])

# Create a custom dataset class

class ButterflyDataset(Dataset):

    def __init__(self, data):
        self.data = data

    def __len__(self):
        return len(self.data)

    def __getitem__(self, index):

        row = self.data.iloc[index]

        image_path = os.path.join(
            "train",
            row["filename"]
        )

        image = Image.open(
            image_path
        ).convert("RGB")

        image = transform(image)

        label = class_to_idx[row["label"]]

        return image, label

# Create training and testing datasets

train_dataset = ButterflyDataset(train_df)

test_dataset = ButterflyDataset(test_df)

```

```

# Create data loaders

train_loader = DataLoader(
    train_dataset,
    batch_size=16,
    shuffle=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=16,
    shuffle=False
)

print("\nTraining Batches:", len(train_loader))
print("Testing Batches:", len(test_loader))

# Display sample butterfly images

images, labels = next(iter(train_loader))

plt.figure(figsize=(10, 8))

for i in range(min(9, len(images))):

    plt.subplot(3, 3, i + 1)

    image = images[i].permute(1, 2, 0)

    image = image * torch.tensor(
        [0.229, 0.224, 0.225]
    ) + torch.tensor(
        [0.485, 0.456, 0.406]
    )

    image = torch.clamp(image, 0, 1)

    plt.imshow(image)

    plt.title(
        classes[labels[i].item()]
    )

```

```
plt.axis("off")

plt.tight_layout()
plt.show()

# Function to create pretrained models

def get_model(name):

    if name == "AlexNet":

        model = models.alexnet(
            weights=models.AlexNet_Weights.DEFAULT
        )

        model.classifier[6] = nn.Linear(
            model.classifier[6].in_features,
            len(classes)
        )

        classifier = model.classifier

    elif name == "VGG16":

        model = models.vgg16(
            weights=models.VGG16_Weights.DEFAULT
        )

        model.classifier[6] = nn.Linear(
            model.classifier[6].in_features,
            len(classes)
        )

        classifier = model.classifier

    elif name == "ResNet50":

        model = models.resnet50(
            weights=models.ResNet50_Weights.DEFAULT
        )

        model.fc = nn.Linear(
```

```

        model.fc.in_features,
        len(classes)
    )

    classifier = model.fc

elif name == "EfficientNetB0":

    model = models.efficientnet_b0(
        weights=models.EfficientNet_B0_Weights.DEFAULT
    )

    model.classifier[1] = nn.Linear(
        model.classifier[1].in_features,
        len(classes)
    )

    classifier = model.classifier

else:

    raise ValueError(
        "Invalid model name"
    )

# Freeze all pretrained layers

for parameter in model.parameters():
    parameter.requires_grad = False

# Allow the new classifier to learn

for parameter in classifier.parameters():
    parameter.requires_grad = True

# Move the model to the selected device

model = model.to(device)

return model

```

```
# List of models used in the assignment

model_names = [
    "AlexNet",
    "VGG16",
    "ResNet50",
    "EfficientNetB0"
]

# Dictionary for storing final accuracy

results = {}

# Train and evaluate every model

for name in model_names:

    print("\n")
    print("Training Model:", name)
    print("\n")

    # Load the pretrained model

    model = get_model(name)

    # Define classification loss

    loss_function = nn.CrossEntropyLoss()

    # Define Adam optimizer

    optimizer = optim.Adam(
        filter(
            lambda parameter: parameter.requires_grad,
            model.parameters()
        ),
        lr=0.001
    )
```

```
# Train the model for three epochs

for epoch in range(3):

    model.train()

    correct = 0
    total = 0
    running_loss = 0

    # Process training batches

    for images, labels in train_loader:

        images = images.to(device)

        labels = labels.to(device)

        # Clear previous gradients

        optimizer.zero_grad()

        # Perform forward propagation

        outputs = model(images)

        # Calculate loss

        loss = loss_function(
            outputs,
            labels
        )

        # Perform backpropagation

        loss.backward()
```

```
# Update classifier weights

optimizer.step()

# Calculate training accuracy

predictions = outputs.argmax(
    dim=1
)

correct += (
    predictions == labels
).sum().item()

total += labels.size(0)

running_loss += loss.item()

# Calculate average training loss

average_loss = (
    running_loss / len(train_loader)
)

# Calculate training accuracy

train_accuracy = (
    correct / total
)

print(
    "Epoch",
    epoch + 1,
    "Loss:",
    round(average_loss, 4),
    "Training Accuracy:",
    round(train_accuracy * 100, 2),
    "%"
)
```

```
# Evaluate the trained model

model.eval()

correct = 0
total = 0

# Disable gradient calculation during testing

with torch.no_grad():

    for images, labels in test_loader:

        images = images.to(device)

        labels = labels.to(device)

        # Generate predictions

        outputs = model(images)

        predictions = outputs.argmax(
            dim=1
        )

        # Count correct predictions

        correct += (
            predictions == labels
        ).sum().item()

        total += labels.size(0)

# Calculate test accuracy

test_accuracy = (
    correct / total
)

# Store model accuracy
```

```

    results[name] = test_accuracy

    print(
        "\n",
        name,
        "Test Accuracy:",
        round(test_accuracy * 100, 2),
        "%"
    )

# Display final comparison

print("\n")
print("FINAL MODEL COMPARISON")
print("\n")

for name, accuracy in results.items():

    print(
        name,
        ":",
        round(accuracy * 100, 2),
        "%"
    )

# Plot model comparison

plt.figure(figsize=(9, 5))

plt.bar(
    results.keys(),
    results.values()
)

plt.xlabel("Model")

plt.ylabel("Test Accuracy")

plt.title(
    "Transfer Learning Model Comparison"
)

```

```
plt.ylim(0, 1)

plt.show()

# Display final results as percentages

print("\n")
print("FINAL RESULTS")
print("\n")

for name, accuracy in results.items():

    print(
        f"{name}: {accuracy * 100:.2f}%"
    )
```

Output :

- **Distinct classes**

Classes: ['ATALA', 'BROWN SIPROETA', 'CRECENT', 'MOURNING CLOAK', 'SLEEPY ORANGE']
Training images: 347
Testing images: 149
Number of classes: 5

- **Samples of Different Tomato leaves dataset**



- **Training DenseNet, VGG16, ResNet50, ResNet101:**

```
Training: DenseNet121
Downloading: "https://download.pytorch.org/models/densenet121-a639ec97.pth" to C:\Users\Akshada/.cache/torch/hub/checkpoints\densenet121-a639ec97.pth
100%|██████████| 30.8M/30.8M [00:16<00:00, 2.01MB/s]
Epoch 1 | Loss: 1.3173 | Accuracy: 47.55 %
Epoch 2 | Loss: 0.6787 | Accuracy: 91.64 %
Epoch 3 | Loss: 0.4294 | Accuracy: 95.68 %

DenseNet121 Test Accuracy: 91.95 %
Training: VGG16
Downloading: "https://download.pytorch.org/models/vgg16-397923af.pth" to C:\Users\Akshada/.cache/torch/hub/checkpoints\vgg16-397923af.pth
100%|██████████| 528M/528M [08:35<00:00, 1.07MB/s]
Epoch 1 | Loss: 1.1668 | Accuracy: 74.64 %
Epoch 2 | Loss: 0.5393 | Accuracy: 95.39 %
Epoch 3 | Loss: 1.1783 | Accuracy: 95.97 %

VGG16 Test Accuracy: 90.6 %
Training: ResNet50
Downloading: "https://download.pytorch.org/models/resnet50-11ad3fa6.pth" to C:\Users\Akshada/.cache/torch/hub/checkpoints/resnet50-11ad3fa6.pth
100%|██████████| 97.8M/97.8M [00:20<00:00, 4.91MB/s]
Epoch 1 | Loss: 1.2949 | Accuracy: 73.78 %
Epoch 2 | Loss: 0.7668 | Accuracy: 96.54 %
Epoch 3 | Loss: 0.516 | Accuracy: 97.69 %

ResNet50 Test Accuracy: 95.97 %
Training: ResNet101
Downloading: "https://download.pytorch.org/models/resnet101-cd907fc2.pth" to C:\Users\Akshada/.cache/torch/hub/checkpoints/resnet101-cd907fc2.pth
100%|██████████| 171M/171M [00:34<00:00, 5.25MB/s]
Epoch 1 | Loss: 1.2147 | Accuracy: 64.55 %
Epoch 2 | Loss: 0.6368 | Accuracy: 92.22 %
Epoch 3 | Loss: 0.3723 | Accuracy: 97.98 %

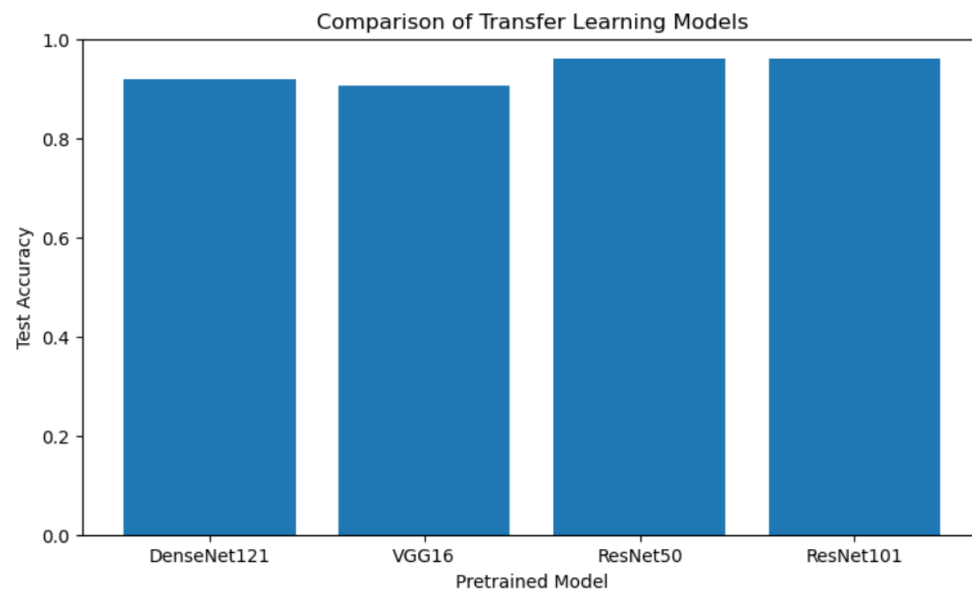
ResNet101 Test Accuracy: 95.97 %
```

- **Model Comparison**

FINAL MODEL COMPARISON

DenseNet121	: 91.95%
VGG16	: 90.60%
ResNet50	: 95.97%
ResNet101	: 95.97%

- **Comparison of Transfer Learning Models**



- **Model Accuracy :**

Model Accuracy Comparison

DenseNet121 -> 91.95%

VGG16 -> 90.60%

ResNet50 -> 95.97%

ResNet101 -> 95.97%